

The Normal model

José G. Dias

Table of contents

1. Definition.....	1
2. Maximum likelihood estimation.....	3
3. Maximum likelihood estimating using R.....	4
4. Bayesian specification	5
4.1 Gamma distribution	6
4.2 Independent priors	7
4.3 Conjugate priors	8
4.4 Model estimation with conjugate priors.....	8
4.5 Model estimation with conjugate priors using R.....	9
4.6 Model estimation with conjugate priors using the MCMC approach.....	11
4.7 Comparing the results.....	16

This text covers the normal distribution used to model continuous data. First, we define the model and its visualization and revise the estimation of the parameters μ and σ^2 using ML. Then, we focus on the Bayesian paradigm. In particular, we study the different specifications of priors and estimation by sampling methods.

1. Definition

The **normal distribution**, also known as the Gaussian distribution or bell curve, is a continuous probability distribution that is symmetrical around its mean (average) value. The probability density function (pdf) of the normal distribution is given by:

$$f(x | \mu, \sigma) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2},$$

where $x \in (-\infty, \infty)$ is the random variable, $f(x | \mu, \sigma)$ is the pdf, μ is the mean (average) of the distribution and σ is the standard deviation, which measures the spread or dispersion of the distribution.

The normal distribution is often denoted as $N(\mu, \sigma^2)$, where N stands for normal, μ is the mean, and σ^2 is the variance (which is the square of the standard deviation).

The shape of the normal distribution is characterized by its bell-shaped curve, and about 68% of the density falls within one standard deviation of the mean, 95% within two standard deviations, and 99.7% within three standard deviations. The normal distribution is widely used in statistics and probability theory due to its mathematical properties and its frequent occurrence in natural phenomena.

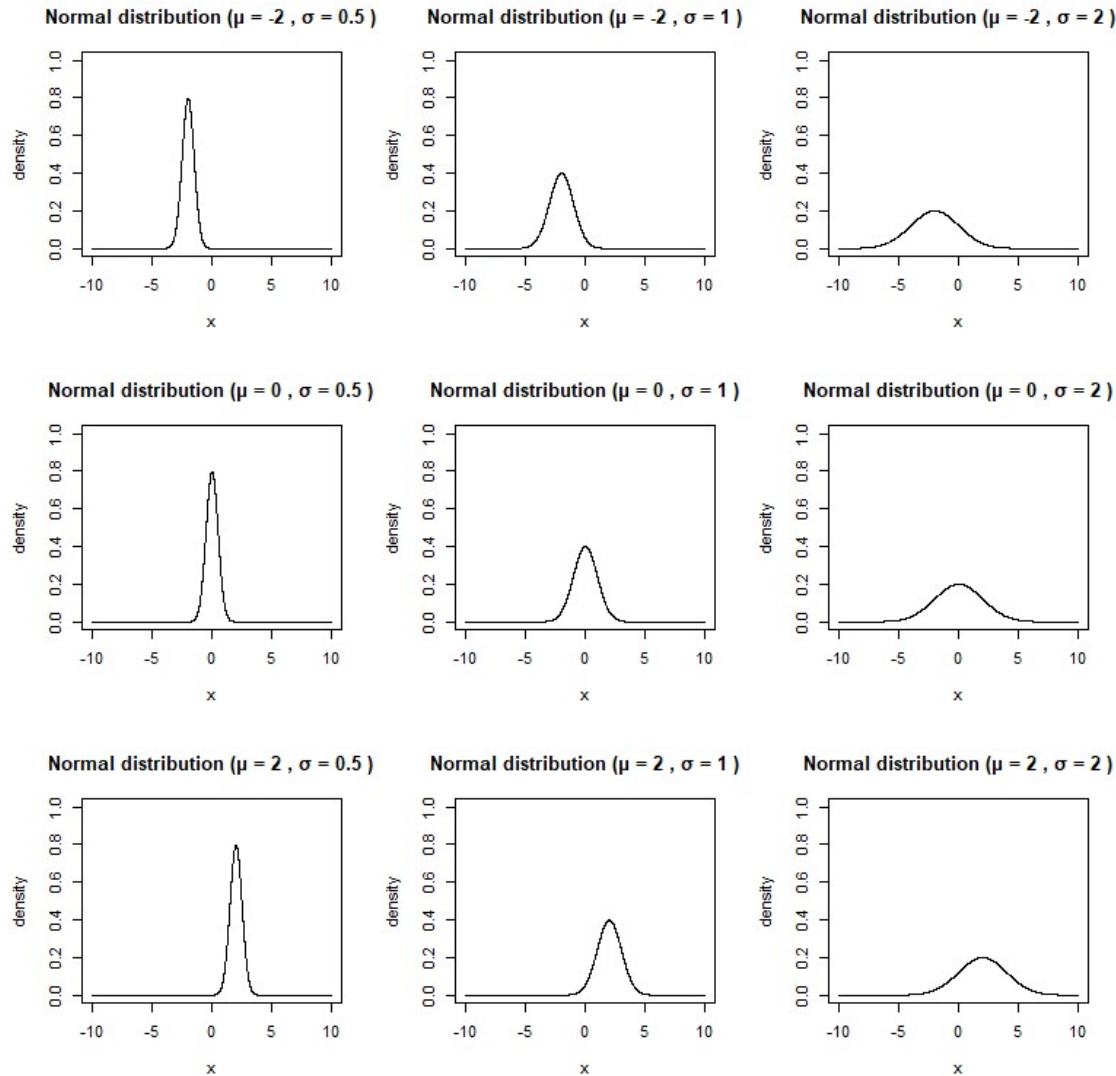
Figure below shows a 3×3 plot with columns vary the $\mu \in (-1,0,1)$ and the rows vary the standard deviation $\sigma \in (0.5,1,2)$ using the following code:

```
# Plot beta distribution for different parameter values
```

```
mu_values <- c(-2, 0, 2)
sigma_values <- c(0.5, 1, 2)

par(mfrow = c(3, 3))

for (i in 1:length(mu_values))
  for (j in 1:length(sigma_values)){
    xx <- seq(-10, 10, length.out = 5000)
    yy <- dnorm(xx, mu_values[i], sigma_values[j])
    tit <- paste("Normal distribution ( $\mu$  =", mu_values[i], ",  $\sigma$  =",
sigma_values[j], ")")
    plot(xx,yy,type='l',xlim = c(-10, 10), ylim = c(0, 1),
        ylab='density',xlab='x',main = tit)
  }
```



Reset plotting layout

```
par(mfrow = c(1, 1))
```

2. Maximum likelihood estimation

The maximum likelihood (ML) estimator for the mean (μ) and standard deviation (σ) of a normal distribution are obtained by maximizing the likelihood function. For a sample of independent and identically distributed observations $(x_1, x_2, \dots, x_n)$ from a normal distribution with mean μ and variance σ^2 , the likelihood function is given by:

$$L(\mu, \sigma^2 | x_1, x_2, \dots, x_n) = \prod_{i=1}^n f(x_i | \mu, \sigma^2) = \prod_{i=1}^n \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x_i - \mu}{\sigma}\right)^2}$$

The log-likelihood function is the natural logarithm of the likelihood function:

$$\ell(\mu, \sigma^2 | x_1, x_2, \dots, x_n) = \sum_{i=1}^n \log f(x_i | \mu, \sigma^2) = -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^n (x_i - \mu)^2$$

To find the ML estimators, we differentiate the log-likelihood function with respect to μ and σ^2 , set the derivatives equal to zero, and solve for the parameters.

The ML estimator for the mean ($\hat{\mu}$) is the sample mean:

$$\hat{\mu} = \frac{1}{n} \sum_{i=1}^n x_i$$

The ML estimator for the variance ($\widehat{\sigma^2}$) is the (not corrected) sample variance:

$$\widehat{\sigma^2} = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2,$$

where $\bar{x}$ is the sample mean. These ML estimators provide the parameter values that maximize the likelihood of observing the given sample under the assumed normal distribution.

3. Maximum likelihood estimating using R

In R, we can use the mean and var functions to obtain the maximum likelihood estimators for the mean and variance, respectively. For example, the next code simulates 1000 observations from the normal distribution with $\mu = 2$ and $\sigma^2 = 1$.

```
# Generate a sample from a normal distribution
```

```
set.seed(123) # For reproducibility  
n <- 1000  
sample_data <- rnorm(n, mean = 2, sd = 1) # standard normal
```

And the ML estimates are:

```
# Maximum Likelihood Estimator for the mean
```

```
mean_estimator <- mean(sample_data)
```

```
# Maximum Likelihood Estimator for the variance
```

```
var_estimator <- var(sample_data) * (n-1)/n
```

```
# Print the estimators
```

```
cat("ML Estimator for Mean:    ", mean_estimator, "\n")
```

```
ML Estimator for Mean:      2.074626
```

```
cat("ML Estimator for Variance:", var_estimator, "\n")
```

```
ML Estimator for Variance: 0.8187336
```

You can use the `hist()` function to create a histogram of the data and then overlay the normal curve using the estimated mean and variance.

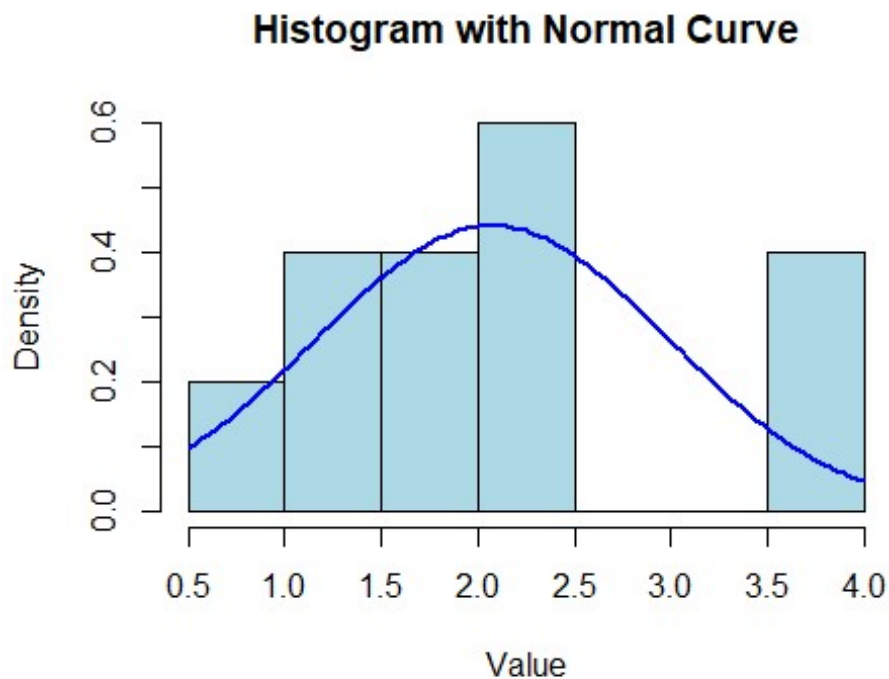
```
sd_estimator <- sqrt(var_estimator)

# Plot histogram with normal curve

hist(sample_data, probability = TRUE, main = "Histogram with Normal Curve",
      xlab = "Value", ylab = "Density", col = "lightblue", border = "black")

# Add normal curve using ML estimators

curve(dnorm(x, mean = mean_estimator, sd = sd_estimator),
      add = TRUE, col = "blue", lwd = 2)
```



4. Bayesian specification

The Bayesian specification of the model, for μ and σ^2 unknowns, is given by the likelihood and prior distribution, i.e.,

$$\pi(\mu, \sigma^2 | x) \propto \pi(x | \mu, \sigma^2) \pi(\mu, \sigma^2).$$

In Bayesian statistics quite often we define the precision $\tau = \sigma^{-2}$, i.e., the inverse of the variance: high precision, low variance.

The next section we discuss the gamma distribution, then different assumptions regarding the prior distribution.

4.1 Gamma distribution

The **gamma distribution** is a continuous probability distribution with positive domain ($x \geq 0$) and the probability density function (pdf) is given by:

$$f(x | \alpha, \beta) = \frac{1}{\beta^\alpha \Gamma(\alpha)} x^{\alpha-1} e^{-\beta x}, \text{ where } \alpha > 0 \text{ and } \beta > 0 \text{ are the shape and scale parameters.}$$

The mean is α/β . The gamma distribution is often denoted by $\text{Gamma}(\alpha, \beta)$.

Figure below shows a 3×3 matrix plot, with the columns vary the shape $\alpha \in (1, 2, 5)$ and with the rows vary the scale $\beta \in (1, 2, 5)$ using the following code:

```
# Plot gamma distribution for different parameter values
```

```
alpha_values <- c(1, 2, 5) #shape
```

```
beta_values <- c(1, 2, 5) #scale
```

```
par(mfrow = c(3, 3))
```

```
for (i in 1:length(alpha_values))
```

```
  for (j in 1:length(beta_values)){
```

```
    xx <- seq(0, 10, length.out = 5000)
```

```
    yy <- dgamma(xx, alpha_values[i], beta_values[j])
```

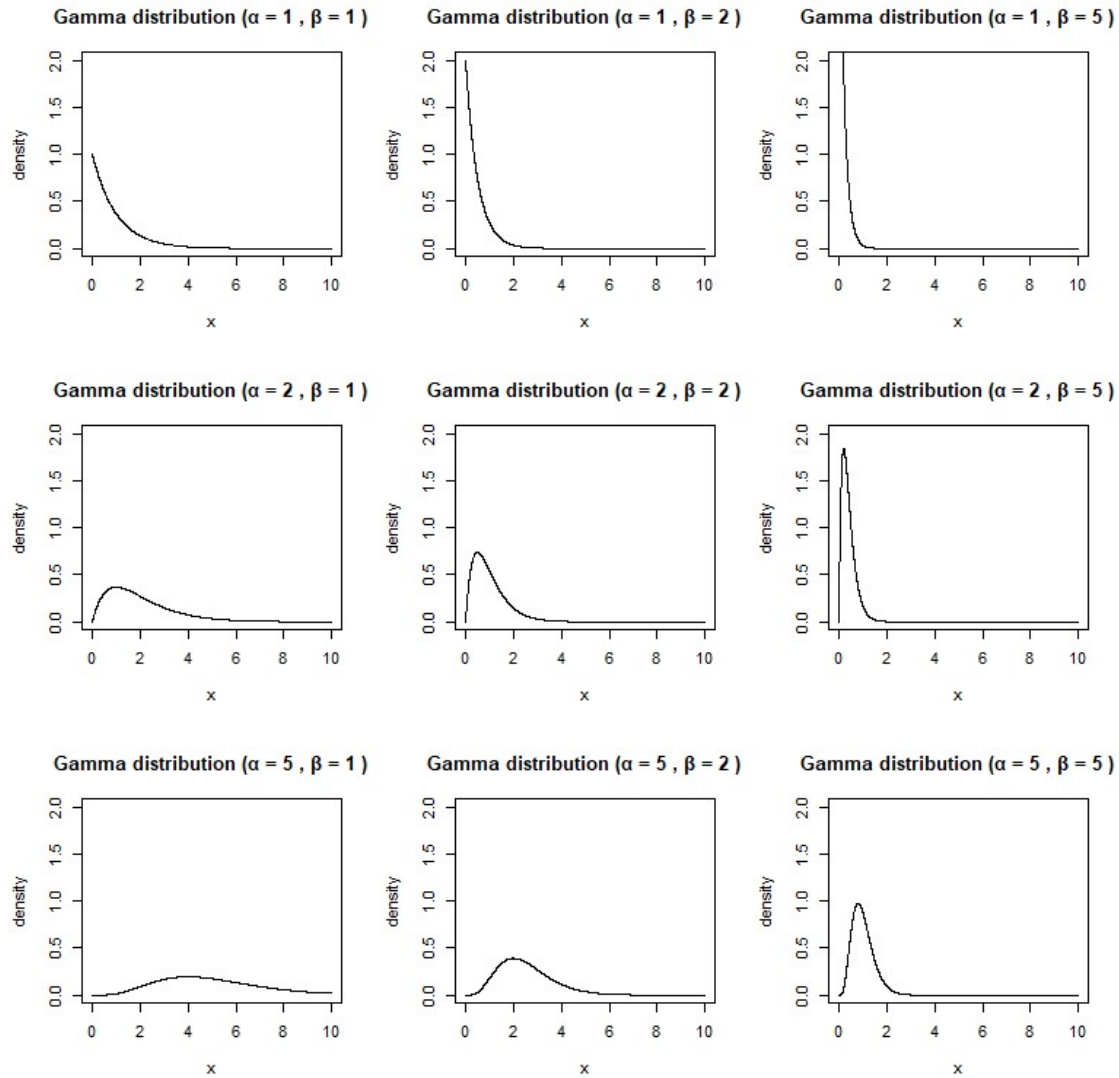
```
    tit <- paste("Gamma distribution ( $\alpha$  =", alpha_values[i], ",  $\beta$  =",
```

```
beta_values[j], ")")
```

```
    plot(xx, yy, type='l', xlim = c(0, 10), ylim = c(0, 2),
```

```
          ylab='density', xlab='x', main = tit)
```

```
  }
```



Reset plotting layout

```
par(mfrow = c(1, 1))
```

4.2 Independent priors

The likelihood and the prior distributions for μ and τ can be expressed as follows:

$\pi(\mu, \tau) = \pi(\mu)\pi(\tau)$. Thus, in this case the specification is:

1. Likelihood: With $x_i \sim N(x|\mu, \tau^{-1})$, then $L(x|\mu, \tau^{-1}) = \prod_{i=1}^n \frac{\sqrt{\tau}}{\sqrt{2\pi}} e^{-\frac{\tau}{2}(x_i - \mu)^2}$
2. Prior for mean μ : $\mu \sim N(0, \sigma_\mu^2)$
3. Prior for precision τ : $\tau \sim \text{Gamma}(1, 1)$

In this case, the gamma distribution with shape and rate parameters both set to 1 is depicted in the graph above.

4.3 Conjugate priors

The posterior distribution combines the likelihood and the priors. This Bayesian model makes inference about the mean (μ) and precision (τ) of the underlying normal distribution using the joint posterior $\pi(\mu, \tau|x)$.

Assuming as before a normal likelihood $p(x|\mu, \tau)$, a normal prior for the mean $p(\mu|\tau)$, and a gamma prior for the precision $p(\tau)$:

$$x_i|\mu, \tau \sim N(\mu, \tau^{-1}) \text{ i.i.d.}$$

$$\mu|\tau \sim N(\mu_0, \lambda_0\tau)$$

$$\tau \sim \text{Gamma}(\alpha_0, \beta_0)$$

4.4 Model estimation with conjugate priors

The posterior distribution for μ and τ can be derived from the likelihood and prior distributions:

$$p(\mu, \tau|x) \propto p(x|\mu, \tau)p(\mu|\tau)p(\tau).$$

It can be shown that $p(\mu, \tau|x)$ is a normal-inverse-gamma (NIG) distribution:

$$p(\mu, \tau|x) \sim \text{NIG}(\mu_n, \lambda_n, \alpha_n, \beta_n),$$

where the parameters of the posterior distribution are updated based on the data and prior information:

$$\mu_n = \frac{\lambda_0\mu_0 + n\bar{x}}{\lambda_0 + n}$$

$$\lambda_n = \lambda_0 + n$$

$$\alpha_n = \alpha_0 + \frac{n}{2}$$

$$\beta_n = \beta_0 + \frac{1}{2} \sum_{i=1}^n (x_i - \bar{x})^2 + \frac{\lambda_0 n (\bar{x} - \mu_0)^2}{2(\lambda_0 + n)}$$

The updated parameters μ_n , λ_n , α_n , and β_n determine the mean, precision, shape, and rate parameters of the posterior distribution, respectively. This closed-form solution arises from the conjugacy properties of the normal and gamma distributions. In this case the conditional distributions can also be derived:

$$\mu|\tau, x \sim N(\mu_n, n_n\tau)$$

$$\tau|x \sim \text{Gamma}(\alpha_n, \beta_n)$$

The R code below uses the closed-form solution for the posterior distribution parameters in an NIG model. Closed-form Bayesian solution for the data used before for ML estimates are shown below:

4.5 Model estimation with conjugate priors using R

Function to calculate posterior parameters for normal-inverse-gamma

```
nig_parameters <- function(data, prior_mean, prior_precision,
                             prior_shape, prior_rate) {

  n          <- length(data)
  mean_data <- mean(data)

  # Posterior parameters

  posterior_mean      <- (prior_precision * prior_mean + n * mean_data) /
    (prior_precision + n)
  posterior_precision <- prior_precision + n
  posterior_shape     <- prior_shape + n/2
  posterior_rate      <- prior_rate + 0.5 * sum((data - mean_data)^2) +
    (prior_precision * n * (mean_data - prior_mean)^2)

  /

    (2 * (prior_precision + n))

  list(mean = posterior_mean, precision = posterior_precision,
        shape = posterior_shape, rate = posterior_rate)
}

# Prior parameters

prior_mean      <- 0
prior_precision <- 1
prior_shape     <- 1
prior_rate      <- 1

# Calculate posterior parameters

posterior_params <- nig_parameters(sample_data, prior_mean,
                                   prior_precision, prior_shape, prior_rate)

# Display the results

cat("Posterior mean:      ", posterior_params$mean, "\n")
Posterior mean:      1.886023

cat("Posterior precision:", posterior_params$precision, "\n")
Posterior precision: 11
```

```
cat("Posterior shape:   ", posterior_params$shape, "\n")
```

```
Posterior shape:      6
```

```
cat("Posterior rate:    ", posterior_params$rate, "\n")
```

```
Posterior rate:       7.050064
```

The R code below depicts the the posterior distribution, likelihood, and prior for the mean (μ) using the ggplot2 package.

```
library(ggplot2)
```

```
# Function to calculate probability density function for normal distribution
```

```
dnorm_precision <- function(x, mean, precision) {  
  sd <- sqrt(1 / precision)  
  dnorm(x, mean = mean, sd = sd)  
}
```

```
# Function to plot prior, likelihood, and posterior for mean
```

```
plot_prior_likelihood_posterior <- function(data, prior_params,  
posterior_params) {  
  prior_df <- data.frame(x = seq(-5, 10, length.out = 1000),  
                        y = dnorm_precision(seq(-5, 10, length.out = 1000),  
                                           prior_params$mean,  
prior_params$precision),  
                        distribution = "Prior")
```

```
  likelihood_df <- data.frame(x = seq(-5, 10, length.out = 1000),  
                             y = dnorm_precision(seq(-5, 10, length.out =  
1000),  
                                                mean(data), 1/length(data)),  
                             distribution = "Likelihood")
```

```
  posterior_df <- data.frame(x = seq(-5, 10, length.out = 1000),  
                             y = dnorm_precision(seq(-5, 10, length.out =  
1000),  
                                                posterior_params$mean,  
posterior_params$precision),  
                             distribution = "Posterior")
```

```
df <- rbind(prior_df, likelihood_df, posterior_df)
```

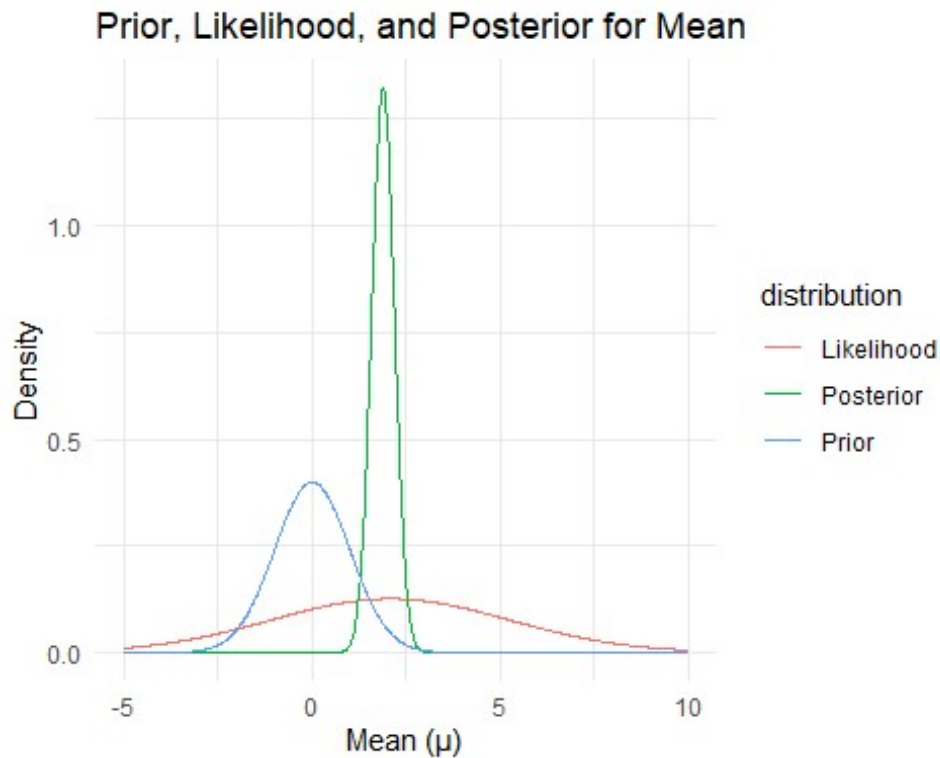
```
ggplot(df, aes(x, y, color = distribution)) +  
  geom_line() +  
  labs(title = "Prior, Likelihood, and Posterior for Mean",  
       x = "Mean ( $\mu$ )", y = "Density") +
```

```
    theme_minimal()
}
```

Then, the plot is:

```
# Plot the distributions
```

```
plot_prior_likelihood_posterior(sample_data, list(mean = prior_mean,
precision = prior_precision), posterior_params)
```



In particular, the posterior mean is:

```
# Display the results
```

```
cat("Posterior Mean:", posterior_params$mean, "\n")
```

```
Posterior Mean: 1.886023
```

4.6 Model estimation with conjugate priors using the MCMC approach

The code below estimates the mean (μ) and precision (τ) parameters in an NIG model. The stan code is defined into blocks:

1. the *data block* contains the observed data;
2. the *parameters block* declares the parameters to be estimated;
3. the *model block* contains the likelihood and priors; and

4. the *generated quantities* block calculates the standard deviation (σ) from the estimated precision (τ).

The rstan package in R compiles and runs the MCMC estimation. In this case, we have Stan code for the normal-inverse-gamma (NIG) model with a normal likelihood and normal and inverse-gamma priors for the mean and precision, respectively.

```
# Install and load the rstan package
# install.packages("rstan")
```

```
library(rstan)
```

Loading required package: StanHeaders

rstan version 2.32.6 (Stan version 2.32.2)

For execution on a local, multicore CPU with excess RAM we recommend calling `options(mc.cores = parallel::detectCores())`.

To avoid recompilation of unchanged Stan programs, we recommend calling `rstan_options(auto_write = TRUE)`

For within-chain threading using ``reduce_sum()`` or ``map_rect()`` Stan functions,

change ``threads_per_chain`` option:

```
rstan_options(threads_per_chain = 1)
```

Do not specify `'-march=native'` in `'LOCAL_CPPFLAGS'` or a Makevars file

```
# Stan model for estimating parameters of the normal distribution
```

```
stan_code <- '
data {
  int<lower=0> N;      // Number of observations
  vector[N] X;       // Data
}
```

```
parameters {
  real mu;           // Mean parameter
  real<lower=0> tau;  // Precision parameter
}
```

```
model {
  // Likelihood
  X ~ normal(mu, sqrt(1/tau));

  // Prior for mean
  mu ~ normal(0, sqrt(1/tau));

  // Prior for precision
  tau ~ gamma(1, 1);
}
```

```

}

generated quantities {
  real<lower=0> sigma;  // Standard deviation

  // Calculate standard deviation from precision
  sigma = sqrt(1 / tau);
}
'

# Compile the Stan model

stan_model <- stan_model(model_code = stan_code)

# Your data

your_data <- list(N = length(sample_data), X = sample_data)

# Run the Stan model

stan_fit <- sampling(stan_model, data = your_data, chains = 4,
                     iter = 2000, warmup = 1000)

```

```

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).
Chain 1:
Chain 1: Gradient evaluation took 1.5e-05 seconds
Chain 1: 1000 transitions using 10 leapfrog steps per transition would take
0.15 seconds.
Chain 1: Adjust your expectations accordingly!
Chain 1:
Chain 1:
Chain 1: Iteration:    1 / 2000 [  0%] (Warmup)
Chain 1: Iteration:   200 / 2000 [ 10%] (Warmup)
Chain 1: Iteration:   400 / 2000 [ 20%] (Warmup)
Chain 1: Iteration:   600 / 2000 [ 30%] (Warmup)
Chain 1: Iteration:   800 / 2000 [ 40%] (Warmup)
Chain 1: Iteration:  1000 / 2000 [ 50%] (Warmup)
Chain 1: Iteration:  1001 / 2000 [ 50%] (Sampling)
Chain 1: Iteration:  1200 / 2000 [ 60%] (Sampling)
Chain 1: Iteration:  1400 / 2000 [ 70%] (Sampling)
Chain 1: Iteration:  1600 / 2000 [ 80%] (Sampling)
Chain 1: Iteration:  1800 / 2000 [ 90%] (Sampling)
Chain 1: Iteration:  2000 / 2000 [100%] (Sampling)
Chain 1:
Chain 1: Elapsed Time: 0.009 seconds (Warm-up)
Chain 1:                0.008 seconds (Sampling)
Chain 1:                0.017 seconds (Total)
Chain 1:

```

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 2).

Chain 2:

Chain 2: Gradient evaluation took 3e-06 seconds

Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 0.03 seconds.

Chain 2: Adjust your expectations accordingly!

Chain 2:

Chain 2:

Chain 2: Iteration: 1 / 2000 [0%] (Warmup)

Chain 2: Iteration: 200 / 2000 [10%] (Warmup)

Chain 2: Iteration: 400 / 2000 [20%] (Warmup)

Chain 2: Iteration: 600 / 2000 [30%] (Warmup)

Chain 2: Iteration: 800 / 2000 [40%] (Warmup)

Chain 2: Iteration: 1000 / 2000 [50%] (Warmup)

Chain 2: Iteration: 1001 / 2000 [50%] (Sampling)

Chain 2: Iteration: 1200 / 2000 [60%] (Sampling)

Chain 2: Iteration: 1400 / 2000 [70%] (Sampling)

Chain 2: Iteration: 1600 / 2000 [80%] (Sampling)

Chain 2: Iteration: 1800 / 2000 [90%] (Sampling)

Chain 2: Iteration: 2000 / 2000 [100%] (Sampling)

Chain 2:

Chain 2: Elapsed Time: 0.013 seconds (Warm-up)

Chain 2: 0.009 seconds (Sampling)

Chain 2: 0.022 seconds (Total)

Chain 2:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 3).

Chain 3:

Chain 3: Gradient evaluation took 4e-06 seconds

Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 0.04 seconds.

Chain 3: Adjust your expectations accordingly!

Chain 3:

Chain 3:

Chain 3: Iteration: 1 / 2000 [0%] (Warmup)

Chain 3: Iteration: 200 / 2000 [10%] (Warmup)

Chain 3: Iteration: 400 / 2000 [20%] (Warmup)

Chain 3: Iteration: 600 / 2000 [30%] (Warmup)

Chain 3: Iteration: 800 / 2000 [40%] (Warmup)

Chain 3: Iteration: 1000 / 2000 [50%] (Warmup)

Chain 3: Iteration: 1001 / 2000 [50%] (Sampling)

Chain 3: Iteration: 1200 / 2000 [60%] (Sampling)

Chain 3: Iteration: 1400 / 2000 [70%] (Sampling)

Chain 3: Iteration: 1600 / 2000 [80%] (Sampling)

Chain 3: Iteration: 1800 / 2000 [90%] (Sampling)

Chain 3: Iteration: 2000 / 2000 [100%] (Sampling)

Chain 3:

Chain 3: Elapsed Time: 0.009 seconds (Warm-up)

Chain 3: 0.009 seconds (Sampling)

Chain 3: 0.018 seconds (Total)
Chain 3:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 4).

Chain 4:

Chain 4: Gradient evaluation took 4e-06 seconds

Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 0.04 seconds.

Chain 4: Adjust your expectations accordingly!

Chain 4:

Chain 4:

Chain 4: Iteration: 1 / 2000 [0%] (Warmup)

Chain 4: Iteration: 200 / 2000 [10%] (Warmup)

Chain 4: Iteration: 400 / 2000 [20%] (Warmup)

Chain 4: Iteration: 600 / 2000 [30%] (Warmup)

Chain 4: Iteration: 800 / 2000 [40%] (Warmup)

Chain 4: Iteration: 1000 / 2000 [50%] (Warmup)

Chain 4: Iteration: 1001 / 2000 [50%] (Sampling)

Chain 4: Iteration: 1200 / 2000 [60%] (Sampling)

Chain 4: Iteration: 1400 / 2000 [70%] (Sampling)

Chain 4: Iteration: 1600 / 2000 [80%] (Sampling)

Chain 4: Iteration: 1800 / 2000 [90%] (Sampling)

Chain 4: Iteration: 2000 / 2000 [100%] (Sampling)

Chain 4:

Chain 4: Elapsed Time: 0.009 seconds (Warm-up)

Chain 4: 0.008 seconds (Sampling)

Chain 4: 0.017 seconds (Total)

Chain 4:

Print a summary of the Stan fit

`print(stan_fit)`

Inference for Stan model: anon_model.

4 chains, each with iter=2000; warmup=1000; thin=1;

post-warmup draws per chain=1000, total post-warmup draws=4000.

	mean	se_mean	sd	2.5%	25%	50%	75%	97.5%	n_eff	Rhat
mu	1.89	0.01	0.34	1.20	1.66	1.89	2.11	2.56	2507	1
tau	0.85	0.01	0.34	0.32	0.61	0.81	1.05	1.63	2547	1
sigma	1.15	0.01	0.25	0.78	0.97	1.11	1.28	1.78	2391	1
lp__	-8.05	0.03	1.03	-10.81	-8.43	-7.74	-7.33	-7.05	1672	1

Samples were drawn using NUTS(diag_e) at Thu Feb 20 02:14:44 2025.

For each parameter, n_eff is a crude measure of effective sample size, and Rhat is the potential scale reduction factor on split chains (at convergence, Rhat=1).

4.7 Comparing the results

Assuming you have already compiled the Stan model and obtained stan_fit

Extract MCMC samples

```
mcmc_samples <- as.array(stan_fit)
```

Extract posterior parameters from closed-form solution

```
posterior_params_closed_form <- nig_parameters(sample_data, 0, 1, 1, 1)
```

```
df <- data.frame(  
  mu= mcmc_samples[, 1, 1]) # mu, chain:1
```

Basic density plot of first chain

```
ggplot(df, aes(x=mu)) +  
  geom_density(fill = "skyblue", alpha = 0.7) +  
  labs(title = "Posterior distribution from MCMC (chain:1) with means (mle  
and posterior)",  
        x = "Mean ( $\mu$ )", y = "Density") +  
  theme_minimal() +  
  # Add mean lines  
  geom_vline(aes(xintercept=mean(mu)),  
             color="blue", linetype="dashed", linewidth=0.5)+  
  geom_vline(aes(xintercept=1.736449 ),  
             color="red", linetype="dashed", linewidth=0.5)+  
  coord_cartesian(xlim = c(-1.5, 4.5), ylim = c(0, 1.5))
```


Posterior distribution from MCMC (chain:1) with means

